

Programming Assignment 2

Q-Learning on a Tabular Gridworld

Skeleton Code: A Python skeleton file will be provided separately. All function signatures, class definitions and TODO markers are already in place — you only need to fill in the implementations.

1. Background

Q-Learning is a model-free, off-policy Temporal Difference (TD) algorithm that learns an action-value function $Q(s, a)$ directly from interaction with an environment, without requiring a model of the transition dynamics. The core update rule is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \times [r + \gamma \times \max_{a'} Q(s', a') - Q(s, a)]$$

where α is the learning rate, γ is the discount factor, r is the immediate reward, and s' is the next state. When the episode terminates, the bootstrap term $\gamma \times \max Q(s', a')$ is omitted.

In the tabular (full-state) setting, Q is stored as an explicit matrix of shape $(|S|, |A|)$. This is the setting you will work with throughout this assignment. The agent interacts with a Gridworld MDP and must discover the optimal policy entirely through trial and error.

2. The Gridworld Environment

2.1 Grid Layout

The environment is a 5×5 grid. Each cell belongs to one of five types:

Symbol	Type	Colour	Behaviour
S	Start	Blue	Episode begins here
G	Goal	Green	Terminal — positive reward
P	Pit	Red	Terminal — negative reward
W	Wall	Grey	Impassable; agent stays in place
.	Empty	White	Free cell; small step penalty

The default grid used throughout this assignment is shown below. You must use this layout for all required parts.

S	.	.	W	.
.	W	.	W	.
.	W	.	.	.
.	.	P	W	.
.	.	.	.	G

S (top-left) is the start. **G** (bottom-right) is the goal. **P** (row 4, col 3) is a pit. **W** cells form a barrier the agent cannot cross.

2.2 MDP Formulation

State space S: All 25 grid positions, encoded as integers 0–24 (row-major order).

Action space A: Four cardinal directions — UP, DOWN, LEFT, RIGHT.

Transitions: Deterministic. Attempting to move into a wall or outside the grid leaves the agent in its current cell.

Rewards: Determined by the type of the cell the agent moves into (see table above).

Termination: An episode ends when the agent reaches the Goal or falls into a Pit.

Discount factor γ : Use 0.99 as the default.

Tip: The skeleton code already implements the GridWorld class structure — you only need to complete the `step()` method and the agent components.

3. Tasks

The assignment is divided into five parts. A skeleton .py file will be provided with all class and function definitions already in place. Your job is to implement the marked sections. Read all docstrings and TODO comments carefully before writing any code.

Part A — Environment & Agent

A.1 Implement GridWorld.step()

Complete the step method so that:

- If the target cell is out of bounds or a Wall, the agent stays in its current position.
- The reward is determined by the type of the cell the agent ends up in.
- The episode terminates (done = True) when the agent reaches the Goal or a Pit.
- The method returns the tuple (next_state, reward, done).

A.2 Initialise the Q-Table

In the `init` method, initialise `q_table` as a NumPy array of shape `(n_states, n_actions)` filled with zeros.

A.3 Epsilon-Greedy Action Selection

Implement `select_action` using the ϵ -greedy policy

A.4 Q-Learning Update Rule

Implement `update_q` using the standard Q-Learning equation shown in Section 1. When the episode is terminal, omit the bootstrap term.

A.5 Epsilon Decay

Implement `decay_epsilon` to multiply `epsilon` by `decay_rate` after each episode, clamping it at `epsilon_min`.

Part B — Training Loop

B.1 Implement train()

Implement the `train` function. For each episode it should:

- Reset the environment to obtain the initial state.
- Loop, selecting actions and stepping through the environment, until the episode terminates.
- Call the agent's update method at every step.
- Decay epsilon at the end of each episode.
- Accumulate and return the list of total rewards, one per episode.

Part C — Convergence Analysis

C.1 Implement `check_convergence()`

Implement :

- Compute a rolling average of episode rewards over the last window episodes.

C.2 Learning Curve & Written Analysis

In your written report, include:

- A plot showing episode reward and the rolling average over training.
- The approximate episode at which you consider the agent to have converged and how you determined this.
- A short explanation (3–5 sentences) of why the reward curve is noisy early in training and stabilises later.

Part D — Visualisation

D.1 Policy Visualisation

Implement - Extract the greedy policy $\pi^*(s) = \operatorname{argmax}_a Q(s, a)$ for every non-wall cell and display it as a grid using Matplotlib, with:

- Arrow glyphs ($\wedge$ $\vee$ $<$ $>$) for free cells showing the best action.
- Distinct markers or colours for the Goal, Pit and Wall cells.
- A descriptive title and clear visual style.

D.2 Value Function Heatmap

Implement - Compute $V(s) = \max_a Q(s, a)$ for all states and display it as a colour heatmap with:

- A suitable diverging or sequential colormap.
- The numerical value annotated inside each cell.
- A colorbar and descriptive title.

4. Submission Guidelines

4.1 What to Submit

- A written report in PDF format (maximum 3 pages) covering Parts C and D.
- .py file with complete code.

4.2 Code Requirements

- Only NumPy and Matplotlib may be used. Do not import any RL libraries (e.g.- Gym, Stable-Baselines, RLlib).
- Do not change function or class signatures.

4.3 Academic Integrity

All submitted code must be your own work. Discussion of high-level concepts with peers is encouraged, but sharing or copying code is not permitted. Any violation will result in a zero for the assignment. Late submission will be penalised.